

BÀI THỰC HÀNH

LAB 18

CAN Loopback Communication

Truyền nhận CAN nội bộ với EduFramework

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	CAN trong hệ thống automotive	2
2.2	Classical CAN và cấu trúc data frame	2
2.3	CAN ID, phân xử bus và phát hiện lỗi	3
2.4	Internal Loopback	3
3	Thiết lập phần cứng	4
3.1	Phần cứng sử dụng	4
3.2	Giao diện sử dụng	4
3.3	Cấu hình nguồn và kết nối	4
4	EduFramework API	4
4.1	CAN_Frame_t	5
4.2	CAN_begin()	5
4.3	CAN_setMode()	5
4.4	CAN_send()	6
4.5	CAN_available() và CAN_read()	6
5	Bài thực hành	7
5.1	Yêu cầu	7
5.2	Chương trình	7
5.3	Kiểm chứng	9
6	Mở rộng	9
6.1	Bài tập mở rộng	10
7	Tài liệu tham khảo	10

1 Giới thiệu

1.1 Tổng quan bài lab

Controller Area Network (CAN) là giao thức truyền thông nối tiếp được phát triển cho các hệ thống điều khiển phân tán, đặc biệt trong lĩnh vực ô tô. Thay vì mỗi bộ điều khiển điện tử (Electronic Control Unit - ECU) phải có đường truyền riêng đến mọi ECU khác, nhiều thiết bị có thể trao đổi thông điệp qua một mạng CAN chung [1] [2].

Bài lab này giới thiệu Classical CAN và cách tạo, truyền, nhận một CAN frame bằng CAN API của EduFramework trên S32K144. Chế độ Internal Loopback cho phép bộ điều khiển FlexCAN nhận lại chính frame đã truyền, nhờ đó chương trình được kiểm chứng trên một board mà không cần ECU thứ hai [6].

Kết quả truyền và nhận được hiển thị trên Serial Monitor. Bài thực hành tập trung vào các khái niệm CAN ID, định dạng frame, độ dài dữ liệu và payload; giao tiếp qua CAN bus vật lý giữa hai ECU sẽ được thực hiện ở Lab 19.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả vai trò của CAN trong mạng truyền thông giữa các ECU của phương tiện.
2. Phân biệt CAN ID tiêu chuẩn 11-bit, mở rộng 29-bit và giới hạn dữ liệu của Classical CAN.
3. Nhận biết các thành phần chính của CAN data frame và nguyên lý ưu tiên thông điệp.
4. Giải thích nguyên lý Internal Loopback và phạm vi kiểm chứng của chế độ này.
5. Sử dụng các CAN API của EduFramework để truyền, nhận và đối chiếu dữ liệu trên Serial Monitor.

2 Kiến thức nền

2.1 CAN trong hệ thống automotive

Một phương tiện có thể sử dụng nhiều ECU để đảm nhiệm các chức năng như quản lý động cơ, truyền động, phanh hoặc hiển thị thông tin. Các ECU cần trao đổi trạng thái và dữ liệu điều khiển mà không làm tăng quá mức số lượng dây kết nối. CAN được sử dụng như một phương thức truyền thông chung giữa các nút trên mạng [1] [2].

CAN là giao thức hướng thông điệp (message-oriented). Mỗi thông điệp mang một identifier (CAN ID), không phải địa chỉ đích cố định của một thiết bị. Trên một CAN bus thông thường, các nút có thể quan sát thông điệp được phát lên bus; ứng dụng hoặc bộ điều khiển quyết định dữ liệu nào cần xử lý [4].

Ở lớp vật lý, CAN tốc độ cao sử dụng hai đường tín hiệu vi sai `CAN_H` và `CAN_L` để truyền dữ liệu giữa các nút thông qua CAN transceiver. CAN controller trong MCU xử lý giao thức, còn transceiver chuyển đổi tín hiệu giữa bộ điều khiển và đường bus [2].

2.2 Classical CAN và cấu trúc data frame

Classical CAN hỗ trợ hai định dạng định danh: Standard CAN sử dụng CAN ID 11-bit và Extended CAN sử dụng CAN ID 29-bit. Với data frame của Classical CAN, trường dữ liệu có độ dài từ 0 đến 8 byte [3].

Định dạng	CAN ID	Đặc điểm
Standard	11-bit	Miền identifier từ 0x000 đến 0x7FF .
Extended	29-bit	Miền identifier từ 0x00000000 đến 0x1FFFFFFF .

Bảng 18.2.1. Hai định dạng identifier trong Classical CAN

Một CAN data frame bao gồm các trường phục vụ đồng bộ, định danh thông điệp, mang dữ liệu và phát hiện lỗi. Các trường quan trọng được tóm tắt trong bảng sau [3] [4] .

Trường	Vai trò
SOF	Đánh dấu bắt đầu một CAN frame.
Arbitration	Chứa identifier và thông tin phục vụ phân xử quyền truy cập bus.
Control	Chứa thông tin điều khiển, bao gồm DLC xác định độ dài trường dữ liệu đối với Classical CAN.
Data	Chứa từ 0 đến 8 byte payload của data frame.
CRC	Cho phép phát hiện lỗi trong dữ liệu frame đã truyền.
ACK	Cho phép nút nhận trên bus xác nhận việc thu một frame hợp lệ.
EOF	Đánh dấu kết thúc CAN frame.

Bảng 18.2.2. Các trường chính của Classical CAN data frame

Trong EduFramework, ứng dụng chỉ cần cung cấp `id` , `format` , `length` và `data[]` . Những trường giao thức như SOF, CRC hay ACK được CAN controller xử lý bên dưới lớp ứng dụng [6] .

2.3 CAN ID, phân xử bus và phát hiện lỗi

Khi nhiều nút cùng muốn phát thông điệp, CAN sử dụng cơ chế phân xử không phá hủy dữ liệu (non-destructive arbitration) dựa trên identifier. Trong trường hợp cùng loại data frame, identifier có giá trị số nhỏ hơn có mức ưu tiên cao hơn; nút thua phân xử ngừng phát và chờ bus rảnh để thử lại [3] [4] .

CAN còn có các cơ chế phát hiện lỗi, bao gồm kiểm tra CRC, kiểm tra bit và kiểm tra định dạng frame. Trên bus thông thường, trường ACK cho phép bên phát biết có ít nhất một nút khác tiếp nhận frame hợp lệ, nhưng ACK không đảm bảo ứng dụng ở ECU đích đã xử lý dữ liệu [3] [4] .

Ghi chú

Các cơ chế arbitration, CRC và ACK được giới thiệu để hiểu nền tảng giao thức. Bài Loopback không kiểm chứng quá trình phân xử giữa nhiều ECU hoặc khả năng hoạt động của lớp vật lý CAN.

2.4 Internal Loopback

Ở chế độ Normal, dữ liệu được phát qua giao diện CAN và có thể được nhận bởi các nút khác trên bus. Internal Loopback tạo đường nhận nội bộ trong CAN controller: frame được truyền sẽ quay trở lại khối nhận của chính bộ điều khiển, không cần đi qua đường `CAN_H` và `CAN_L` bên ngoài [6] .

Trong EduFramework, `CAN_setMode(CAN_MODE_LOOPBACK)` kích hoạt chế độ này. Bài thực hành có thể kiểm tra việc tạo frame, gửi, nhận và đọc hàng đợi dữ liệu trên một MCU. Kết quả `PASS` chứng minh các trường đã so sánh khớp nhau trong thử nghiệm nội bộ; kết quả đó không thay thế kiểm thử trên CAN bus thực.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài lab sử dụng một MaaZEDU Development Board có vi điều khiển S32K144 và FlexCAN0. Chương trình truyền nhận trong chế độ Internal Loopback nên không cần ECU thứ hai, CAN transceiver rời hay dây nối CAN_H / CAN_L.

Phần cứng	Mô tả
MaaZEDU Development Board	Board S32K144 dùng để chạy chương trình CAN Loopback.
12V DC Adapter	Nguồn ngoài theo yêu cầu vận hành CAN của MaaZEDU.
USB Cable	Kết nối máy tính để nạp chương trình và sử dụng Serial Monitor.

Bảng 18.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Giao diện sử dụng

EduFramework sử dụng FlexCAN0 của S32K144. Trên MaaZEDU, CAN0_TX được ánh xạ tới PTE5 và CAN0_RX tới PTE4 theo tài liệu board [7] [5].

Tín hiệu	Chân MCU	Vai trò
CAN0_TX	PTE5	Đường truyền từ CAN controller tới transceiver ở chế độ Normal.
CAN0_RX	PTE4	Đường nhận từ transceiver về CAN controller ở chế độ Normal.
Serial1	LPUART1	Hiển thị dữ liệu và kết quả kiểm chứng trên Serial Monitor.

Bảng 18.3.2. Các giao diện liên quan tới Lab 18

Hai chân CAN được nêu nhằm nhận biết phần cứng; bài Loopback không cần kết nối chúng bằng dây. Không nối tắt CAN_H với CAN_L, vì đây là hai đường riêng của giao tiếp CAN vi sai [2].

3.3 Cấu hình nguồn và kết nối

Theo MaaZEDU Development Board Guide, chức năng CAN trên board yêu cầu nguồn ngoài [7]. Bài lab áp dụng cấu hình đã được kiểm thử trên MaaZEDU: chuyển jumper chọn nguồn sang chế độ nguồn ngoài, cấp nguồn bằng adapter 12V và duy trì kết nối USB để nạp chương trình, sử dụng Serial Monitor.

Ghi chú

Tắt nguồn trước khi thay đổi jumper. Cấp nguồn adapter vào đúng đầu nối nguồn của board; không cấp 12V trực tiếp vào chân MCU hoặc chân tín hiệu. Internal Loopback không đòi hỏi dây nối CAN ngoài. Yêu cầu nguồn ngoài là đặc điểm cấu hình MaaZEDU dùng trong bài thực hành.

4 EduFramework API

EduFramework cung cấp Arduino-style CAN API trên lớp FlexCAN driver. Ứng dụng không cần cấu hình trực tiếp Message Buffer, ngắt nhận hoặc hàng đợi; những thành phần đó được quản lý ở các tầng bên dưới [6].

4.1 CAN_Frame_t

`CAN_Frame_t` biểu diễn một Classical CAN frame ở mức ứng dụng. Các trường dữ liệu được sử dụng trong bài lab gồm:

Trường	Ý nghĩa
<code>id</code>	CAN identifier của thông điệp.
<code>format</code>	Định dạng ID: <code>CAN_STANDARD</code> hoặc <code>CAN_EXTENDED</code> .
<code>length</code>	Số byte payload, từ 0 đến 8.
<code>data[]</code>	Mảng byte chứa payload, tối đa <code>CAN_MAX_DATA_LENGTH</code> .

Bảng 18.4.1. Các thành phần của `CAN_Frame_t`

Bài lab chọn `id = 0x100`, `format = CAN_STANDARD`, `length = 1` và đặt dữ liệu truyền tại `data[0]`. Identifier `0x100` là giá trị minh họa do bài lab chọn, không phải mã thông điệp tốc độ xe quy định chung cho mọi hãng.

4.2 CAN_begin()

CAN_begin

Khởi tạo bus CAN với tốc độ bit được chỉ định; chế độ ban đầu là Normal.

Cú pháp

```
CAN_begin(bitRate);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>bitRate</code>	CAN bitrate	Tốc độ truyền theo bit mỗi giâygiây.

Giá trị trả về

`true` nếu khởi tạo thành công; `false` nếu khởi tạo thất bại hoặc bitrate không hợp lệ.

Giá trị `500000UL` tương ứng `500 kbit/s` và được sử dụng trong ví dụ CAN 2.0 của NXP [5].

4.3 CAN_setMode()

CAN_setMode

Chọn chế độ hoạt động của CAN controller sau khi đã khởi tạo.

Cú pháp

```
CAN_setMode(mode);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>mode</code>	CAN operating mode	Bài lab sử dụng <code>CAN_MODE_LOOPBACK</code> để nhận lại frame nội bộ.

Giá trị trả về

`true` nếu chuyển chế độ thành công; `false` nếu chế độ không hợp lệ hoặc bộ điều khiển chưa sẵn sàng.

`CAN_begin()` và `CAN_setMode()` phải hoàn thành thành công trước khi chương trình thử truyền dữ liệu [6] .

4.4 CAN_send()

CAN_send

Truyền một CAN data frame theo cơ chế blocking, chờ hoàn thành hoặc hết thời gian chờ của driver.

Cú pháp

```
CAN_send(&txFrame);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
frame	CAN Frame	Địa chỉ của <code>CAN_Frame_t</code> chứa thông tin cần truyền.

Giá trị trả về

`true` nếu hoàn thành truyền thành công; `false` nếu truyền thất bại hoặc frame không hợp lệ.

Một frame hợp lệ phải có `format` phù hợp với miền ID và `length` không vượt quá 8 byte [6] .

4.5 CAN_available() và CAN_read()

CAN_available

Kiểm tra hàng đợi CAN có ít nhất một frame đã nhận hay không.

Cú pháp

```
CAN_available();
```

Giá trị trả về

`true` nếu có frame đang chờ đọc; `false` nếu không có dữ liệu hoặc CAN chưa được khởi tạo.

CAN_read

Đọc và lấy ra frame cũ nhất trong hàng đợi nhận theo cơ chế non-blocking.

Cú pháp

```
CAN_read(&rxFrame);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
frame	CAN Frame	Địa chỉ biến <code>CAN_Frame_t</code> dùng để lưu frame được nhận.

Giá trị trả về

`true` nếu đọc thành công một frame; `false` nếu hàng đợi rỗng hoặc tham số không hợp lệ.

Dữ liệu nhận được EduFramework đưa vào hàng đợi thông qua cơ chế ngắt. Ứng dụng kiểm tra `CAN_available()` trước, sau đó dùng `CAN_read()` để lấy frame và xử lý trong vòng lặp chính [6] .

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình CAN Loopback trên một board S32K144. Khởi tạo CAN ở 500 kbit/s và chuyển sang CAN_MODE_LOOPBACK. Mỗi giây, chương trình truyền một Standard CAN frame với ID 0x100, DLC bằng 1 và byte dữ liệu tăng dần.

Sau khi truyền, chương trình đọc frame nhận lại, hiển thị giá trị TX, RX và thông báo PASS khi CAN ID, định dạng, DLC và byte payload trùng khớp. Serial Monitor sử dụng baud rate 9600.

5.2 Chương trình

Trong src/main.c, sử dụng chương trình đã kiểm chứng trên MaaZEDU Development Board:

```
#include "Arduino.h"
#include "can.h"
int main(void)
{
    CAN_Frame_t txFrame = {0};
    CAN_Frame_t rxFrame = {0};
    uint8_t value = 0U;
    setup();
    Serial1_begin(9600U);
    if ((false == CAN_begin(500000UL)) ||
        (false == CAN_setMode(CAN_MODE_LOOPBACK)))
    {
        Serial1_println("CAN initialization failed.");
        while (1) {}
    }
    Serial1_println("=== CAN Loopback Demo ===");
    txFrame.id = 0x100UL;
    txFrame.format = CAN_STANDARD;
    txFrame.length = 1U;
    while (1)
    {
        txFrame.data[0] = value;
        if (true == CAN_send(&txFrame))
        {
            Serial1_print("TX: ");
            Serial1_printInt(value);
            Serial1_println("");
            if (true == CAN_available())
            {
                if (true == CAN_read(&rxFrame))
                {
                    Serial1_print("RX: ");
                    Serial1_printInt(rxFrame.data[0]);
                    Serial1_println("");
                    if ((txFrame.id == rxFrame.id) && (txFrame.format == rxFrame.format) &&
                        (txFrame.length == rxFrame.length) &&
                        (txFrame.data[0] == rxFrame.data[0]))
                    {
                        Serial1_println("Result: PASS");
                    }
                    else
                    {
                        Serial1_println("Result: FAIL");
                    }
                }
            }
        }
        else
        {
            Serial1_println("CAN transmission failed.");
        }
        Serial1_println("-----");
        value++;
        delay(1000U);
    }
}
```

`setup()` khởi tạo môi trường EduFramework và `Serial1_begin(9600U)` chuẩn bị Serial Monitor. Sau khi `CAN_begin(500000UL)` hoàn tất, chương trình gọi `CAN_setMode(CAN_MODE_LOOPBACK)` để nhận nội bộ các frame đã truyền.

`txFrame` được cấu hình với Standard CAN ID `0x100` và độ dài payload một byte. Trong vòng lặp, `value` được gán vào `txFrame.data[0]` rồi gửi bằng `CAN_send()`. Nếu truyền thành công và có frame trong hàng đợi, `CAN_read()` lấy dữ liệu vào `rxFrame`. Chương trình so sánh bốn trường `id`, `format`, `length` và `data[0]` để xác định kết quả `PASS` hoặc `FAIL`.

Biến `value` thuộc miền `0..255` và tăng sau mỗi lần lặp. Khi vượt `255`, giá trị trở về `0` theo cách biểu diễn số nguyên không dấu 8-bit. Lệnh `delay(1000U)` tạo khoảng nghỉ một giây giữa hai lần thử.

5.3 Kiểm chứng

Sau khi đặt jumper ở chế độ nguồn ngoài và cấp adapter `12V`, nạp chương trình vào MaaZEDU Development Board. Mở Serial Monitor tại `9600` baud và quan sát giá trị TX/RX theo từng chu kỳ.

Kết quả đã kiểm chứng trên phần cứng:

```
=== CAN Loopback Demo ===
TX: 0
RX: 0
Result: PASS
-----
TX: 1
RX: 1
Result: PASS
-----
TX: 2
RX: 2
Result: PASS
-----
```

Tiếp tục quan sát một số chu kỳ kế tiếp để xác nhận dữ liệu tăng đều và các trường được so sánh luôn khớp nhau. Trong chương trình này, việc không có frame sẵn sàng tại đúng thời điểm `CAN_available()` được gọi sẽ khiến chu kỳ đó không in kết quả `RX` hoặc `PASS`; do đó việc kiểm chứng dựa trên những frame thực tế đã được đọc và đối chiếu, không chỉ dựa vào thông báo truyền thành công.

KẾT QUẢ MONG ĐỢI

FlexCAN0 được khởi tạo và chuyển sang Internal Loopback thành công. Với các frame đã nhận lại, CAN ID `0x100`, định dạng Standard, DLC bằng `1` và byte dữ liệu RX khớp giá trị TX. Serial Monitor hiển thị `Result: PASS` qua các chu kỳ liên tiếp mà không cần kết nối CAN bus bên ngoài.

6 Mở rộng

Bài thực hành chính sử dụng giao tiếp blocking và đọc dữ liệu bằng polling trên hàng đợi nhận. EduFramework còn có các API và chế độ hoạt động phục vụ ứng dụng CAN nhiều chức năng hơn [6].

API / Chế độ	Chức năng
<code>CAN_sendNonBlocking()</code>	Khởi động truyền CAN mà không chờ thao tác truyền kết thúc.
<code>CAN_isTxBusy()</code>	Kiểm tra hiện có một lần truyền non-blocking đang hoạt động hay không.
<code>CAN_isTxComplete()</code>	Kiểm tra cờ hoàn tất thành công của lần truyền gần nhất.
<code>CAN_onReceive()</code>	Đăng ký callback được gọi từ ngữ cảnh ngắt khi có frame đã được đưa vào hàng đợi nhận.
<code>CAN_MODE_NORMAL</code>	Chế độ giao tiếp CAN bình thường qua bus vật lý.
<code>CAN_MODE_LISTEN_ONLY</code>	Chế độ theo dõi lưu lượng CAN mà không chủ động phát frame.
<code>CAN_EXTENDED</code>	Sử dụng CAN identifier mở rộng 29-bit.
<code>CAN_end()</code>	Dừng hoạt động CAN và đặt lại trạng thái của lớp CAN API.

Bảng 18.6.1. Các chức năng CAN mở rộng của EduFramework

6.1 Bài tập mở rộng

Điều chỉnh frame của chương trình để mang hai byte dữ liệu thay vì một byte. Sau mỗi lần truyền, kiểm tra cả hai byte của payload nhận được trước khi in `PASS`. Giữ nguyên chế độ Internal Loopback và không cần thiết bị bổ sung.

Các chức năng truyền non-blocking, callback nhận và chế độ Normal sẽ được vận dụng trong Lab 19 khi hai ECU trao đổi dữ liệu qua CAN bus vật lý.

7 Tài liệu tham khảo

- [1] J. A. Cook, J. S. Freudenberg, *Controller Area Network (CAN)*, University of Michigan, EECS 461 Lecture Notes, 2008.
- [2] Steve Corrigan, Texas Instruments, *Introduction to the Controller Area Network (CAN)*, SLOA101B, Rev. B, 2016.
- [3] CAN in Automation (CiA), *CAN CC (Classical CAN)*, CiA CAN Knowledge.
- [4] Kvaser, *The CAN Bus Protocol Tutorial*, Kvaser Technical Tutorial.
- [5] NXP Semiconductors, *S32K1xx Series Cookbook*, AN5413, Rev. 5, 2020.
- [6] EduFramework, *Arduino-style CAN API (can.h, can.c)*, EduFramework Source Code.
- [7] FPT Software, *MaaZ Edu Development Board User Manual*, Rev. 1.0, 2025.